Editorial
European Junior Olympiad in Informatics 2024

Chișinău, Moldova

16-22 August 2024

Contents

1 Many Pairs

Problem author: Ovidiu Rața.

Solution in $O(n^3 + n^2k)$

Since n and k are small, we can use various methods to solve the problem. The following solution has a time complexity of $O(n^3 + n^2k)$ and a space complexity of $O(n + k)$.

We proceed by iterating through every node of the tree. For each node u, we consider every pair of edges adjacent to it, denoted as (e_1, e_2). Since there are $n-1$ edges in total, there are at most $O(n^2)$ such pairs.

Let v and t be the nodes at the ends of edges e_1 and e_2, respectively, excluding node u. Starting from node u, we perform a graph traversal (either BFS or DFS), ignoring all adjacent nodes except for v and t. During each traversal, we keep track of the visited nodes.

After the traversal, we iterate through each of the initial k pairs and check if both nodes associated with a pair have been visited. We compute the sum of the values C_i for such valid pairs and select the maximum possible sum. This maximum sum is the answer for node u.

Solution in $O(n(n + k))$

We iterate through every node u in our tree, root the tree at u, and start a DFS procedure from u. We use the DFS to determine, for every node, which child of u its subtree belongs to.

While performing the DFS, we notice that there are two possible types of pairs:

(1) Pairs where both endpoints are in the same subtree of a child of u.

(2) Pairs that are split between the subtrees of two different children of u.

We iterate through every pair and check which case applies. If both endpoints are in the same subtree, we add the value of that pair to the overall sum of the pairs contained within that particular subtree of a child of u. If the pair is split between two subtrees, we add its value to a two-dimensional array sum, where $sum_{i,j}$ represents the sum of all pairs where one endpoint is in the subtree of child i of u, and the other endpoint is in the subtree of child j.

We then iterate through every pair of children of u and compute the answer obtained when keeping the edges corresponding to that pair. This can be achieved by using the sums and the array described above.

The overall complexity of this algorithm is $O(nk + n^2)$. The first term represents the

complexity of performing our DFS, and the second term is the amortized complexity for iterating through every pair of children for the root of our DFS.

Solution in $O(n \log n + k \log n)$

Let us root the tree at vertex 1. First, we perform some precomputations necessary for finding Lowest Common Ancestors (LCAs). We start a DFS which will help us answer our queries. For the vertex u that we are currently processing during our DFS, we should maintain the following information:

(1) All the pairs for which one vertex is within the subtree of one child of u, and the other vertex is within the subtree of another child of u.

(2) For each child of u, the sum of the values of the pairs which are completely contained within the subtree of that child.

(3) All the pairs for which one vertex is contained within the subtree of u, and the other vertex is outside of the subtree rooted at u.

(4) The sum of all the pairs completely disjoint with the subtree of u.

The answer for a vertex u can be computed based on the above information. There are a few cases to consider:

- **Option 1:** Keep the edge to the parent of u and an edge to a child of u. In this case, we combine the information from points 2, 3, and 4 above. We sum the values from points 2 and 4, and include the contributions from all pairs (A, B, C) where u lies on the path from A to B. We can compute this by maintaining a prefix sum from leaves to root on the tree where we add C to nodes A and B, and subtracting $2 \cdot C$ at their LCA.
- **Option 2:** Keep the edges towards two children of u. In this case, we need to take the two largest values from point 2 above.
- **Option 3:** Consider pairs where one vertex is within the subtree of one child of u, and the other vertex is within the subtree of another child of u. We combine the information from points 1 and 2 above. Normally, this would take $O(\deg(u)^2)$ time, but we can optimize by noticing that a pair (A, B) will have A in one subtree of u and B in another subtree only if u is their LCA. Therefore, we can precompute, for each such pair (A, B), their contribution at the LCA u, and in total perform $O(k)$ additional computations amortized.

The answer for vertex u is the maximum among these three options. We continue our DFS for the other nodes.

Overall time complexity: $O(n \log n + k \log n)$.

2 Cheese

Problem author: Cheng Zhong.

Let's begin by defining transactions. Let P represent the price array of all N farmers. A formal way to describe an exchange is given by the equation:

$$P_j - P_i = A + kB, k \in \mathbb{Z}$$

In other words,

$$P_j - P_i \equiv A \pmod{B}$$

While $B = -1$ is a special case, for generality, we assign B a value of $2^{34} > N \cdot A$. Since the difference between any two prices is at most $N \cdot A$, this choice of modulo ensures no collisions between different P values.

Farmers can be represented as nodes in a graph, and the exchanges between them as edges. The task is to check whether a new edge can be added to the current graph while maintaining its validity.

Clearly, if no path exists between nodes i and j, there is no relationship between them, and nothing affects the validity of adding the edge.

Now, suppose there is a path connecting the two nodes. Let the modulos of the edges in this path be denoted as $b_1, b_2, \ldots, b_k$.

The resulting relationship between the two nodes is:

$$P_j - P_i = a' \pmod{b}$$

where

$$b = \min(b_1, b_2, \ldots, b_k)$$

We can easily compute a' using modular arithmetic over b. When multiple paths exist, they must all be consistent. We select the path with the largest b. The condition for the edge to be valid is:

$$A \equiv a' \pmod{min(B, b)}$$

Disjoint Set Union

Let's assume all edges in the graph share the same modulo. In this case, the graph can be stored as a disjoint set union (DSU). We are only concerned about the relationship between P_i and P_j and whether the new edge is consistent with previous information. This modified DSU structure helps address both concerns.

Consider two nodes i and j with a common root r.(If i and j are not part of the same set, we can make no assertions about $P_j - P_i$.) Using existing or easily computable values of $P_r - P_i \pmod{b}$ and $P_r - P_j \pmod{b}$, we derive:

$$P_j - P_i = -(P_r - P_j) + (P_r - P_i) \pmod{b}$$

Final Algorithm

The above principle forms the basis of our final solution. Since edges have different moduli, but because moduli are powers of 2, we can process the edges in descending order of powers. Hence, for any given modulo B, there can be at most M edges.

For each edge, to check if adding it will preserve graph validity, we iterate over all powers of 2, from lowest to highest, verifying the consistency of the DSU at each power. If the edge is found to be invalid, i.e., it introduces a contradiction, we flag it as such. Otherwise, we update the DSU of all divisors to include the new edge.

By utilizing path compression and small-to-large optimization, we achieve a time complexity of $O((M + N) \log B)$.

3 Old Orhei

Problem author: Victor Purice.

Test group 1, $Q = 1$

In this test group, we can solve the problem using the following procedure:

- At the beginning we set the position to be S.
- Iterate from L to R and for every index i:
  - If $X_{A_i} = S$ then at this step we will move to node Y_{A_i} and therefore set new value $S := Y_{A_i}$
  - If $X_{A_i} \neq S$ then we do nothing.

At the end we just output value S.

Test group 2, $N = 2$

In this test group, we have only two nodes which means that there can exist 4 types of edges: from node 1 to node 1, from node 1 to node 2, from node 2 to node 1 and from node 2 to node 2. The edges from node 1 to node 1 and from node 2 to node 2 are not that important since they don't change the state of our current node.

We keep track of all the indexes i with $1 \leq i \leq T$ such that $X_{A_i} \neq Y_{A_i}$ and store them in a set. When we receive a query on an interval $[l, r]$ we find the biggest index in this set which is also inside the interval. If there doesn't exist such an index, we print S. Otherwise let's denote the found index by I, and we print Y_{A_I}.

Keeping the set described above will take $O(Q \log T)$ time complexity. An update will correspond to possibly deleting an element from the set and inserting another one.

Test group 3, $M = N - 1, X_i = i, Y_i = i + 1$

In this case Y_i is always greater than X_i, hence the algorithm in the first group will have S non-decreasing. In particular, condition $X_{A_i} = S$ will be satisfied at most N times. This means that instead of iterating over all the indices, we can immediately move to the next smallest unprocessed index j in the interval $[L, R]$ such that $X_{A_j} = S$.

We store N sets, with set indexed i containing all the indices t such that $X_{A_t} = i$. Then, when being in node S and searching for the index j described above we binary search it in the set S. When finding such an index j we set $S = Y_{A_j}$.

Updates will mean deleting and inserting elements into two separate sets and thus will take $O(\log T)$ compared with $O(N \log T)$ for query. The complexity of the algorithm will be $O(QN \log T)$.

Test group 4, no queries of type 2**,** $T \leq 3 \cdot 10^4$

In this case, since we do not have any updates, we can precompute the following table $D[i, j, k]$ which will be the answer for interval $[i, i + 2^k - 1]$ starting value j. The case $k = 0$ is trivial, and for $k > 0$ we have the following recurrence:

$$D[i, j, k] = D[i + 2^{k-1}, D[i, j, k - 1], k - 1]$$

After we've precomputed table D in $O(TN \log T)$, we can answer every query using a similar algorithm as in RMQ (Range Minimum Query).

Test group 5

We will create a similar data structure as before except with segment trees so that updates are also supported. For interval $[l, r]$ in the segment tree, we have that $D[l, r, i]$ is the answer for interval $[l, r]$ is we start at node i. If $l = r$, the answer is trivial, otherwise if $m = \frac{l+r}{2}$, then we have the following recurrence

$$D[l, r, i] = D[m + 1, r, D[l, m, i]]$$

The complexity of the algorithm is $O(QN \log T)$.

4 CF Duels

Problem author: Cheng Zhong.

Test Group 1: All Prizes are 1, Small N

In this test group, all prizes are $p_i = 1$ and $N \leq 10$.

Since winning any match contributes equally to the total prize, we aim to maximize the number of matches won. Due to the small value of N, we can compute the answer using a brute-force approach. The opponent's player assignments do not affect the outcome significantly, so the answer will always be either 0 or -1.

To determine if the answer is 0, we can iterate over all possible permutations of our N players' skill levels and check whether we can win more matches than we lose. The total time complexity for this approach is $O(N! \cdot N)$.

Test Group 2: All Prizes are 1

We can build upon the approach from the first test group but need a more efficient solution than the factorial time complexity. Here, we introduce our first greedy strategy:

- **Winning Matches**: If we can win against opponent i, we assign the unassigned player with the **smallest** skill level that is still sufficient to beat that opponent (i.e., has a skill level at least b_i).
- **Losing Matches**: If we cannot win against opponent i, we assign the unassigned player with the smallest overall skill level.

This strategy ensures we maximize the number of wins while using our players efficiently. We can implement this using a set data structure, resulting in a time complexity of $O(N \cdot \log N)$.

Test Group 3: The Answer is Either 0 or 1

In this test group, the answer is always 0 or 1. We need to determine whether the answer is 0, which occurs when, even in the worst-case scenario with no information about the opponent, we can still win.

This subtask introduces our second key idea: **the opponent's optimal assignment**. The opponent will assign their players such that higher-skilled players compete in stadiums with larger prizes. Specifically, if we have two stadiums i and j with $p_i > p_j$, the opponent will ensure $s_i > s_j$.

Given this, we can apply the same greedy strategy from the second test group but with consideration for prize values. To maximize our profit, we process stadiums in decreasing order of prizes and assign our strongest available players to the stadiums with the highest

prizes.

If, after this assignment, we accumulate a greater total prize sum than the opponent, the answer is 0; otherwise, it is 1.

Test Group 4: The Answer is Either -1 or $N-1$

In this test group, the answer is either -1 or $N-1$. We need to determine whether the answer is $N-1$, which occurs when, after fixing the assignments for $N-1$ days, we can ensure victory.

We can utilize the same greedy strategy from the third test group but apply it to a **fixed assignment**. This means that we know the opponent's player assignments for $N-1$ days.

By applying our strategy under these conditions, we can check if it's possible to secure victory after $N-1$ days.

Test Group 5: $N \leq 5$

With $N \leq 5$ in this test group, we can simulate all possible permutations of our players' assignments and check each one. This approach is similar to the first subtask, but we can also incorporate the greedy strategies from test groups 2, 3, and 4 to optimize our solution.

Test group 6: No additional constraints

In this test group, we aim to combine the greedy strategies from all previous subtasks. Initially, this approach yields a solution with a time complexity of $O(N^2 \cdot \log N)$, which is too slow.

However, we observe that the more information we have about the opponent's players, the better we can strategize our assignments. This means that if day k is a possible answer, then day $k-1$ is also possible. Using this insight, we can apply **binary search** to find the **earliest** day where victory is guaranteed.

We check if day k is a possible answer by applying the greedy strategies from test groups 2, 3, and 4. Each check has a complexity of $O(N \cdot \log N)$. Since we use binary search over N days, the total time complexity becomes $O(N \cdot (\log N)^2)$.

5 Sweets

Problem author: Alexandru Andrei

Solution for $p_i = i - 1$

The given tree forms a line (a path) in this subtask. It is always optimal to pick node N as the destination.

For $n, q \leq 2000$, we can solve the problem using a straightforward approach: maintain the array l, and for each of the q days, check for every i whether $\sum_{j=1}^{i} l_j \geq t_i$. For larger constraints we need to make a few additional observations.

Let $f(i, j)$ denote the prefix sum of length i of the array l after the first j days. Notice that $f(i, j) \leq f(i, j + 1)$ always holds, meaning the prefix sums are non-decreasing over days. This implies that for each node i, there exists either some day x such that node i is successful for all days $j \geq x$, or node i never becomes successful. Let $good(i)$ denote the earliest day when node i becomes successful. Then, the answer for day j is the number of nodes i such that $good(i) \leq j$, formally expressed as $\sum_{i=1}^{n} [good(i) \leq j]$.

There are multiple ways to solve this subtask optimally. An elegant method is as follows:

Initialize a segment tree that supports range addition and maximum query on an interval. Set the initial value at index i to $-t_i$. Iterate over the days from 1 to q. On day j, add x_j to the interval $[u_j, n]$ in the segment tree. While the maximum value in the segment tree is greater than or equal to 0, find the position pos with the maximum value, set $good(pos) = j$, and update the value at position pos to $-\infty$ in the segment tree to exclude it from future considerations.

By doing this, we find the value of $good(i)$ for all i. Now we sort the array $good$, and compute for each day j the number of nodes where $good(i) \leq j$ in $O(n)$ total time.

The final time complexity of this solution is $O((q + n) \cdot \log n)$.

Other methods to find the array $good$ include parallel binary search, 2D or persistent segment trees, and lazy-propagation segment trees, although they may be less optimal or more complex to implement.

Solution in $O(n \cdot q)$

We process the days from 1 to q, maintaining and updating the array l each day. To find the answer for day j, we perform a Depth-First Search (DFS).

In the DFS, we keep track of a parameter sum, which represents the sum of all l_i values along the path from the root to the current node u. The DFS function for node u should return the maximum number of successful nodes on any path starting from u within its subtree.

Here is the C++ code for the DFS function:

```cpp
int dfs(int u, long long sum) {
    sum += l[u];
    int ret = 0;
    for (int v : g[u]) {
        int q = dfs(v, sum);
        ret = max(ret, q);
    }
    return ret + (sum >= x[u]);
}
```

Note: Since *sum* can exceed the limits of 32-bit integers, it is important to use a 64-bit data type (e.g., `long long`).

By running `dfs(1, 0)` after updating l on day j, we obtain the answer for that day.

Solution for $u_i = 1$ for All Events

When all events occur at the root ($u_i = 1$), finding the value of $good(u)$ becomes straightforward. The problem reduces to determining, for each day, the maximum number of successful nodes along a path from the root to any node.

To find $good(u)$, we can use prefix sums and binary search. Build prefix sums on the array x. For each u, perform a binary search to find the earliest day i such that the prefix sum up to i satisfies $pref_x[i] \geq t_u$. Alternatively, sorting the array of pairs $\{t_u, u\}$ and using a two-pointer algorithm can find $good(u)$ in $O(n)$ time. Both methods result in a time complexity of $O(n \cdot \log n)$.

Next, we need to find, for each day j, the value:

$$\max_u \left(\sum_{v \in \text{path}(1,u)} [good(v) \leq j] \right)$$

where $\text{path}(1, u)$ denotes the set of vertices along the simple path from node 1 to node u.

To compute this efficiently, we use the Euler Tour Technique.

First, perform a DFS from the root to assign entry and exit times to each node, effectively flattening the tree into an array. Then, for each day j from 1 to n, we process nodes with $good(u) = j$. For each such node u, we add 1 to the range corresponding to its subtree in the flattened tree using a segment tree or a Binary Indexed Tree (Fenwick Tree). After updates, the maximum value in the segment tree represents the answer for day j.

Full Solution

We reuse the definitions from earlier: let $f(u, j)$ denote the sum $\sum l_v$ for all vertices v on the path from node u to the root on day j.

Since $f(u, j+1) \geq f(u, j)$ (the sums are non-decreasing over days), it suggests that for each market u, there is a point in time when Sandu becomes successful and remains successful thereafter. We denote this day as $good(u)$—the first day on which Sandu becomes successful in market u. For vertices where Sandu never becomes successful during the q days, we set $good(u) = \infty$.

First Part: Finding $good(u)$

There are several methods to find $good(u)$ for each u, including parallel binary search, persistent segment trees, and 2D segment trees. An effective approach is to maintain a segment tree that supports point updates and, given a value x, can find the smallest index i such that the prefix sum up to i is greater than or equal to x.

Run a DFS starting from the root (vertex 1). Let $S(u)$ denote the set of events at node u, stored as pairs (i, x_i) where i is the day and x_i is the value.

During the DFS:

- On entering node u, for each event in $S(u)$, perform $add(i, x_i)$ in the segment tree.
- After updates, the prefix sum up to index i represents the sum of all l_v values along the path from u to the root after day i.
- Determine $good(u)$ by finding the smallest i such that the prefix sum is greater than or equal to t_u.
- Before backtracking, on exiting node u, for each event in $S(u)$, perform $add(i, -x_i)$ to revert the changes and maintain correctness for other branches.

Second Part: Computing Answers for Each Day

When a node u becomes successful (on day $good(u)$), it contributes 1 to the count for all paths ending in its subtree. We use the Euler Tour Technique to handle this efficiently.

First, flatten the tree by performing a DFS to record entry and exit times for each node, effectively mapping the tree to an array. Then, process the days from 1 to q. For each day i, for each node u with $good(u) = i$, perform a range addition of 1 over its subtree interval in the segment tree. After updates, the maximum value in the segment tree represents the answer for day i.

Final Complexity:

- Time Complexity: $O((q + n) \cdot \log n)$ - Space Complexity: $O(n + q)$

6 Hora

Problem author: Victor Purice.

Test group 1, $N = 34$, $Q_{full} = 34$

In this test group, the intended number of queries is equal to N. We can query every index individually and check every continuous sequence of length K to find the answer.

Observations

In order to continue with the next test groups we need to make some observations. The notations used in the solution are the same as in the problem statement.

Observation 1: The sought absolute difference is always 1 for odd K and 0 for even K.

Proof: Let us consider that every element of our circle which corresponds to a boy has a value of 1 and every element of our circle which corresponds to a girl has a value of -1.

Let us denote with S the sum of every possible sum of continuous length K in our circle. It is easy to see that $S = K \cdot (A_0 + A_1 + \dots A_{N-1})$. However, $(A_0 + A_1 + \dots + A_{N-1}) = 0$, which means $S = 0$ as well. Hence, there exists one interval of length K with sum ≥ 0, and another interval of length K with sum ≤ 0. Let us denote boundaries of these 2 intervals by the tuples (L_1, R_1) and (L_2, R_2), with $L_1 < L_2$. For two consecutive intervals, the difference of their sums is either -2, $+2$, or 0, which means that there exists an interval of length K with its start in the closed interval $[L_1, L_2]$, for which the sum of its elements is either $-1, 0$ or 1 depending if K is odd or even.

Instead of solving the problem for some odd K, we can solve the problem for $K + 1$ and return the exact same answer. This allows us to search for an interval with an absolute difference of 0 instead of 1. Hence, for the remainder of the editorial, we will assume that we are searching for an interval of even length K with absolute difference 0.

In addition, for the remaining of solution, we say that an interval is of type `'+'` if it has more boys than girls, of type `'-'` if it has more girls than boys and of type `'0'` if the number of boys equals the number of girls.

Observation 2: If we have found an interval of type `'+'` and an interval of type `'-'`, then the sought interval of minimal absolute difference can be found with a binary search between these two initial intervals.

Explanation: Let us denote with L the starting index of the left-most interval, and with R the starting index of the right-most interval. Without the loss of generality, we can assume that the interval $[L, L+K-1]$ is of type `'+'` and $[R, R+K-1]$ is of type `'-'`.

Let us take $M = \frac{L+R}{2}$. If we query the interval $[M, M + K - 1]$, we have 3 options:

- The interval is of type `'+'`. In this case, by `Obervation 1` we know that a solution exists with the starting index in the interval $[M, R]$, and hence we can continue our binary search there.
- The interval is of type `'-'`. In this case, by `Obervation 1` we know that a solution exists with the starting index in the interval $[L, M]$, and hence we can continue our binary search there.
- The interval is of type `'0'`. In this case, this is the answer that we were looking for.

Observation 3: The number of intervals of type `'+'` is not necessarily equal to the number of intervals of type `'-'`.

Even though the gut feeling might suggest that the number of intervals of type `'+'` should be approximately equal to the number of intervals of type `'-'`, and a randomized solution for finding the initial interval of type `'+'` and the initial interval of type `'-'` should be sufficient, this is not the case.

Counter-Example:

Let's consider the following counter-example: $N = 400, K = 20$, and our circular string is

$$\underbrace{XX\ldots X}_{11}\underbrace{YY\ldots Y}_{9}\underbrace{XX\ldots X}_{11}\underbrace{YY\ldots Y}_{9}\ldots\underbrace{XX}_{2}\underbrace{YY\ldots Y}_{38}$$

In this example, we have 145 intervals of type `'+'`, 53 intervals of type `'-'`, and 2 intervals of type `'0'`.
In general, in the worst case, there can be approx. $\sqrt{N}$ intervals of type `'+'`, and approx. $N - \sqrt{N}$ intervals of type `'-'` or vice versa.
While a randomized solution might get a decent score, for a full score a deterministic solution with fewer queries exists.

Test group 2, $N = 100000$, all the boys are adjacent to each other, $Q_{full} = 18$

Let's make a call for the interval $[0, K-1]$. If this interval is of type `'0'`, we found our answer. Otherwise, we can assume that this interval is of type `'+'`. Since all the boys are adjacent to each other and all the girls are adjacent to each other as well we can be sure that the interval $[\frac{N}{2}, \frac{N}{2} + K - 1]$ is of type `'-'`. We can hence apply a binary search as described above in `Observation 2`.

Total number of queries: $1 + log_2(\frac{N}{2})$.

Test group 3, $N = 100000$, hora was generated randomly, $Q_{full} = 34$

Since the configuration of the hora was generated randomly, the number of `'+'` intervals is approx. equal to the number of `'-'` intervals. Hence we can randomly query intervals of length K until we find one of type `'+'`, one of type `'-'`, and perform our binary search as described in `Observation 2`.

Test group 4, $N = 100000, K = 50000$, $Q_{full} = 18$

This test group represents the case where $K = \frac{N}{2}$. In this group, we can query the interval $[0, K-1]$. If this interval has absolute difference `'0'`, then we found our answer. Otherwise, without the loss of generality, we can consider that this interval is of type `'+'`. This means that the interval $[K, N-1]$ is of type `'-'` we can perform our binary search as described in `Observation 2`.

Total number of queries: $1 + log_2\left(\frac{N}{2}\right)$.

Test group 5, $N = 65536, K = 128$, $Q_{full} = 26$

This test group represents the case where N and K are both powers of 2. As in the previous cases, we search for an interval of length K of type `'+'` and an interval of length K of type `'-'`.

We start by querying the interval $[0, K-1]$. Let's assume that this interval is a type `'+'` interval. In order to find an `'-'` interval of length K, we can start by making a query on the interval $[0, \frac{N}{2} - 1]$. If this is a `'-'` or `'0'` interval, we can continue our search inside it. Otherwise, if it is an `'+'` interval, we can continue our search on the other half, since it is guaranteed to be a `'-'` interval. We continue this procedure until we arrive at an interval of length K which is guaranteed to be of type `'-'`. Now that we have found an interval of type `'+'` of length K and an interval of type `'-'` of length K we can apply our binary search as in `Observation 2`.

Total number of queries: $1 + log_2\left(\frac{N}{K}\right) + \log_2(N - K + 1)$.

Test group 6, $N = 100000, K = 400$, $Q_{full} = 26$

This test group represents the case where $K|N$. We can solve it in a similar fashion as the previous test group. We start by querying the interval $[0, K-1]$. Let's assume that this interval is of type `'+'`. Now we need to find an internal of type `'-'` of length K.

We divide the total interval $[0, N]$ in two almost equal intervals $[0, M-1]$ and $[M, N-1]$ such that both of them have a length which is divisible by K. We query for the number of boys in the interval $[0, M-1]$. If this interval is of type `'-'`, we continue our search inside it. Otherwise, we continue our search in the interval $[M, N-1]$, as it is guaranteed to be an interval of type `'+'`. After finding the sought interval of type `'-'` of length K, we apply the procedure described in `Obersvation 2`.

Total number of queries: $1 + log_2\left(\frac{N}{K}\right) + log_2(N - K + 1)$.

Test group 7, $N = 100000, K = 99601$, $Q_{full} = 26$

This test group is the *almost* the complement of the previous one.

Since K is odd, we can solve the problem for $K = 99600$ instead. However, in the previous case, we have found a way to solve the problem for $K = 400$, and since $99600 + 400 = 100000$, we can find the solution for $K = 400$, and take its complement, which will have the absolute difference 0 as well.

Total number of queries: $1 + log_2\left(\frac{N}{K}\right) + log_2(N - K + 1)$.

Test group 8, $N = 100000, K = 330$, $Q_{full} = 68$

This test group represents the case where K does not divide N, but, conveniently, $(N \bmod K)|N$. More specifically, $100000 \bmod 330 = 10$. Since $10|100000$, we can solve the problem for $K = 10$ by applying the procedure from the Test group 4. Now, we take the complement of the found interval, which would have length 99990, and solve the problem by applying the procedure from Test group 4 again for $N = 99990, K = 330$. Note that this solution breaks our circular property, but we don't need it when $K \nmid N$. As we will see later, it is useful only for the case when $K|N$.

Total number of queries: $1+log_2\left(\frac{N}{N \bmod K}\right)+log_2(N - N \bmod K + 1)+1+log_2\left(\frac{N}{K}\right)+log_2(N - K + 1)$.

Test group 9, Mixed values for N and K (no additional constraints), $Q_{full} = 34$

Let's take an extended circular string which is constructed by appending our original string to itself until the total length becomes equal to the lowest common multiple between N and K. This new string has the same property as the initial one since the number of boys equals the number of girls. Let's say that we want to query the number of boys on an interval $[L, R]$ in our extended string. It can be easily proved that the answer for this query is the same as the answer for the query $[L \bmod N, R \bmod N]$ in our original string. This means that we can solve the general case on this larger string by applying the same procedure from the test group 4, and we can map the found interval of length K to another interval of length K in our original string.

The total number of queries then would be $1 + \log_2\left(\frac{LCM(N,K)}{K}\right) + log_2(N - K + 1)$.